Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives, contrôle de flux et algorithmique en langage C

Synthèse exécutive : Ce cours présente de manière exhaustive les trois structures itératives fondamentales (while, do-while, for) en algorithmique et en langage C. L'objectif prioritaire est de comprendre comment automatiser la répétition d'instructions sans duplication de code, conformément au principe DRY (Don't Repeat Yourself). La séance détaille les critères d'évaluation des conditions (pré-condition vs post-condition), les contextes d'utilisation précis de chaque boucle, la gestion des variables de contrôle, ainsi que la prévention systématique des boucles infinies et des erreurs de syntaxe fréquentes en contrôle continu.

1. Utilité des structures itératives et principe DRY

En programmation algorithmique et en langage C, la répétition d'instructions ne doit jamais être gérée par du copier-coller. Si l'on doit traiter 10 000 éléments d'un tableau ou redemander une saisie utilisateur invalide, dupliquer manuellement les lignes de code détruit la maintenabilité, alourdit inutilement l'exécutable et constitue une source massive de bogues. Les structures itératives (ou boucles) permettent de répéter l'exécution d'un bloc d'instructions tant qu'une condition logique (expression booléenne) reste vraie. L'enjeu central de la conception d'une boucle repose sur la maîtrise de la condition d'arrêt et l'actualisation effective de la variable de contrôle à chaque itération. Si le corps de la boucle n'altère pas les variables testées dans la condition, celle-ci reste indéfiniment vraie, provoquant une 'boucle infinie' qui bloque le programme.

- Principe DRY (Don't Repeat Yourself) : interdiction fondamentale de dupliquer du code pour répéter une tâche.
- Condition d'arrêt : expression booléenne évaluée pour déterminer la poursuite ou la sortie de la boucle.
- Variable de contrôle / Compteur : variable impérativement mise à jour au fil des itérations.
- Boucle infinie : dysfonctionnement majeur où la condition reste indéfiniment vraie, bloquant le programme et saturant le processeur.

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'attrape et l'emballe (instruction). Si le capteur ne détecte plus de colis ou si le compteur atteint le quota fixé, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide et finirait par surchauffer et détruire la machine.*

2. La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est la boucle de choix lorsque le nombre précis d'itérations n'est pas connu à l'avance et dépend d'un événement dynamique (ex: une saisie utilisateur, la lecture d'un fichier jusqu'à la fin, ou la réception d'un signal réseau). Puisque le test logique s'effectue en amont de l'entrée dans le bloc, si la condition est fausse dès la première évaluation, l'intérieur de la boucle n'est jamais exécuté (0 itération). Pour garantir la sortie de la boucle, il est impératif d'initialiser les variables de contrôle avant la boucle et d'assurer leur modification explicite à l'intérieur du corps de la boucle.

- Pré-conditionnement : évaluation du test logique AVANT chaque exécution du bloc d'instructions.
- Intervalle d'itérations : de 0 à N passages (le bloc peut ne jamais s'exécuter si la condition est fausse d'entrée).
- Initialisation préalable : toute variable testée dans la condition doit posséder une valeur définie avant l'instruction while.

Cas concret enseignant : *Exemple du contrôle du code PIN au distributeur de billets : la machine demande le code TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est inférieur à 3. Si l'utilisateur saisit le bon code du premier coup, la boucle s'arrête immédiatement sans répéter la demande.*

3. La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence de la boucle 'while', la structure 'do-while' garantit que les instructions contenues dans son corps seront exécutées au moins une fois avant le premier test logique. Elle est principalement dédiée à la saisie sécurisée de données (validation de bornes numériques) et à la gestion de menus interactifs. En langage C, sa syntaxe impose un point-virgule après l'expression fermante du `while(condition);`. L'oubli de ce point-virgule constitue l'une des erreurs de compilation les plus fréquentes commises lors des évaluations.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions.
- Garantie d'exécution : la boucle s'exécute obligatoirement au minimum 1 fois (1 à N itérations).
- Syntaxe C stricte : obligation impérative d'apposer un point-virgule ';' après la parenthèse fermante du while : do { ... } while(condition);

Cas concret enseignant : *Cas du contrôle de saisie d'une note d'examen : 'Il est techniquement impossible de vérifier si la saisie faite par un étudiant est valide avant qu'il n'ait tapé au moins une valeur sur le clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure parfaitement adaptée.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en centralisant l'initialisation, le test de passage et la mise à jour de la variable de boucle en une seule ligne d'en-tête. Elle est systématiquement recommandée lorsqu'on doit effectuer un nombre d'itérations fixe et connu à l'avance (ex: parcourir les éléments d'un tableau de taille N ou répéter une opération un nombre déterminé de fois). Bien que les trois expressions séparées par des points-virgules dans l'en-tête soient optionnelles en C, les omettre nuit gravement à la lisibilité et est fortement sanctionné en contrôle de TP/TD.

- En-tête condensé : regroupe l'initialisation, la condition de maintien et l'incrémentation sous la forme for(init; cond; inc).
- Nombre d'itérations déterminé : privilégié lorsque le nombre exact de répétitions est connu avant d'entrer dans la boucle.
- Portée (scope) : possibilité de déclarer la variable d'itération directement au sein de l'en-tête (ex: for(int i = 0; ...)).

Cas concret enseignant : *Métaphore du tour de piste d'athlétisme : 'Vous savez avant de vous lancer que vous devez effectuer exactement 5 tours. Vous initialisez votre compteur au tour 1 (init), vous vérifiez que vous n'avez pas dépassé 5 (cond), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (inc).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Sans l'instruction d'incrémentation (ex: i++), la variable de contrôle 'i' conserve sa valeur initiale. La condition booléenne reste donc indéfiniment vraie, empêchant le programme de sortir de la boucle. Le processeur s'épuise alors à exécuter le bloc d'instructions en boucle infinie, ce qui gèle l'exécution de l'application.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, exactement. Sur le plan de la logique algorithmique, toute boucle 'for' peut être réécrite sous la forme d'un 'while' en plaçant l'initialisation avant la boucle et l'incrémentation à la toute fin du corps de la boucle. Cependant, on utilise le 'for' par convention de lisibilité dès que le nombre d'itérations est connu au préalable.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit obligatoirement valider que N est un entier strictement positif (> 0) à l'aide d'une boucle 'do-while', puis calculer la somme au moyen d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclarer les variables entières n, somme (initialisée à 0) et i. Étape 2 : Mettre en place la saisie sécurisée avec la structure do-while : do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); Étape 3 : Initialiser somme = 0 avant le calcul. Étape 4 : Écrire la boucle for pour effectuer le cumul de 1 jusqu'à N : for (i = 1; i <= n; i++) { somme += i; } Étape 5 : Afficher le résultat avec printf("La somme des %d premiers entiers est %d\n", n, somme);

Résultat : Pour un essai où l'utilisateur saisit d'abord -2 puis 5 : La boucle do-while rejette -2 et réclame une nouvelle saisie. Lorsque 5 est entré, la boucle do-while valide la valeur. La boucle for réalise 5 itérations (i=1 -> somme=1 ; i=2 -> somme=3 ; i=3 -> somme=6 ; i=4 -> somme=10 ; i=5 -> somme=15). Le programme affiche correctement : 'La somme des 5 premiers entiers est 15'.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' est à pré-condition : le test logique a lieu avant l'exécution du bloc (de 0 à N itérations). La boucle 'do-while' est à post-condition : le test logique a lieu après l'exécution du bloc, garantissant au moins 1 exécution.

Q : Que se passe-t-il si la condition d'une boucle 'for' est fausse dès le premier test ?

R : Le corps de la boucle 'for' n'est jamais exécuté (0 itération). Le programme passe directement à l'instruction située immédiatement après la boucle.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule ';' obligatoire après la parenthèse fermante de l'expression while à la fin de la structure do { ... } while(condition);